

Connect your Newrro ESP32 kit to TRUSTMESH

From a box of parts to a live, blockchain-verified sensor station on the website — step by step, with labelled photos of **your** hardware.



⌚ About 45 minutes · 🖥️ Laptop with TRUSTMESH installed · Newrro controller box + sensor board + ribbon cable · USB-C data cable · IR sensor · RC522 RFID reader + tag · optional: flame sensor, DHT22, multimeter

Read this first. The pin functions in this guide were read from the silkscreen in your photos (e.g. *IO15*, *IO16*, *IO17*, *IO18*, *IO3* on the RFID port; *IO45–IO48* ports, which only exist on an **ESP32-S3**). Nothing in this guide has been tested on your physical kit yet — each step includes a quick check so you catch a wrong assumption *before* it matters. Power everything **off** while plugging or unplugging.

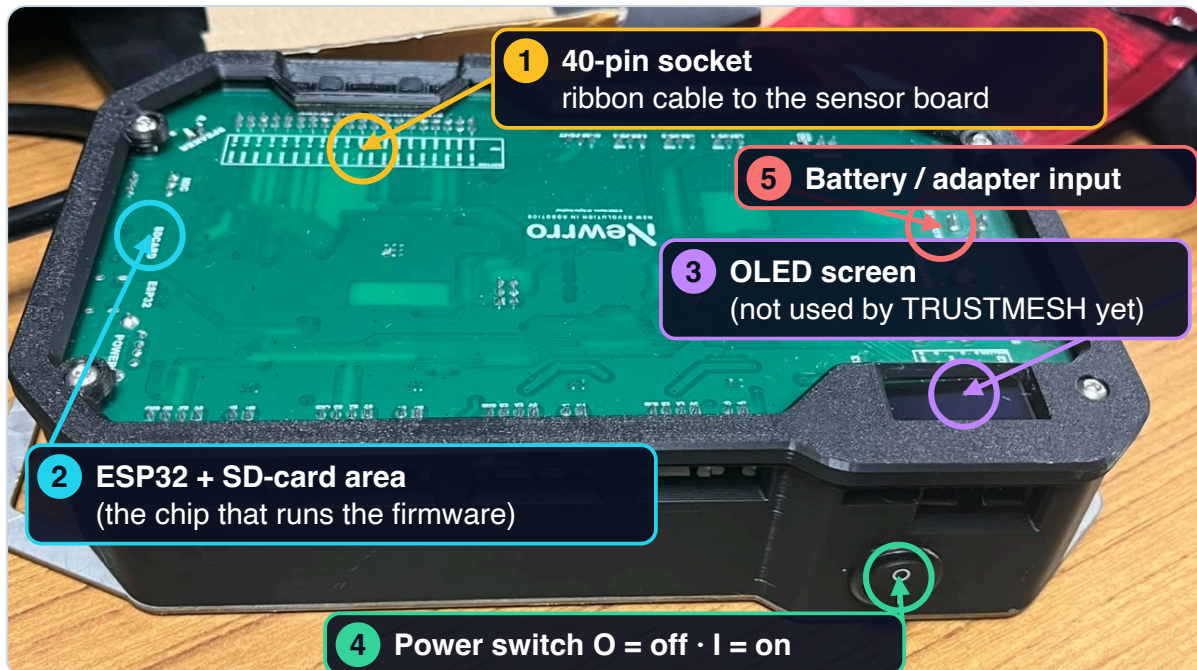
The 9 steps

#	Step	Time
1	Get to know your controller box	3 min
2	Get to know your sensor board	3 min
3	Start TRUSTMESH on the laptop	5 min
4	Plug the sensors into the board	8 min
5	Connect the box to the laptop (USB-C) & identify the chip	5 min
6	Write the device key & flash the firmware	8 min
7	Test every sensor (diagnostic mode)	6 min
8	Link the kit to the website (USB bridge)	3 min
9	Run the Live Demo with real hardware	4 min

STEP 1

Get to know your controller box

This is the **brain**. It contains the ESP32-S3 chip that reads the sensors, **signs every reading with its secret key**, and sends it to the laptop.



Your controller box. The USB-C port used for programming is on the side of the box (not visible in this photo).

Tip: keep the switch at **O (off)** while connecting cables. The USB-C cable powers the board for programming; turn the switch on to use the battery/adapter.



Your ERD USB-C data cable (1 m).

Why a data cable matters

The laptop talks to the ESP32 through this cable: to **upload the firmware** and then to **carry the signed sensor readings** to the website (via the USB bridge program). A charge-only cable gives power but no data — the laptop would never see the board.

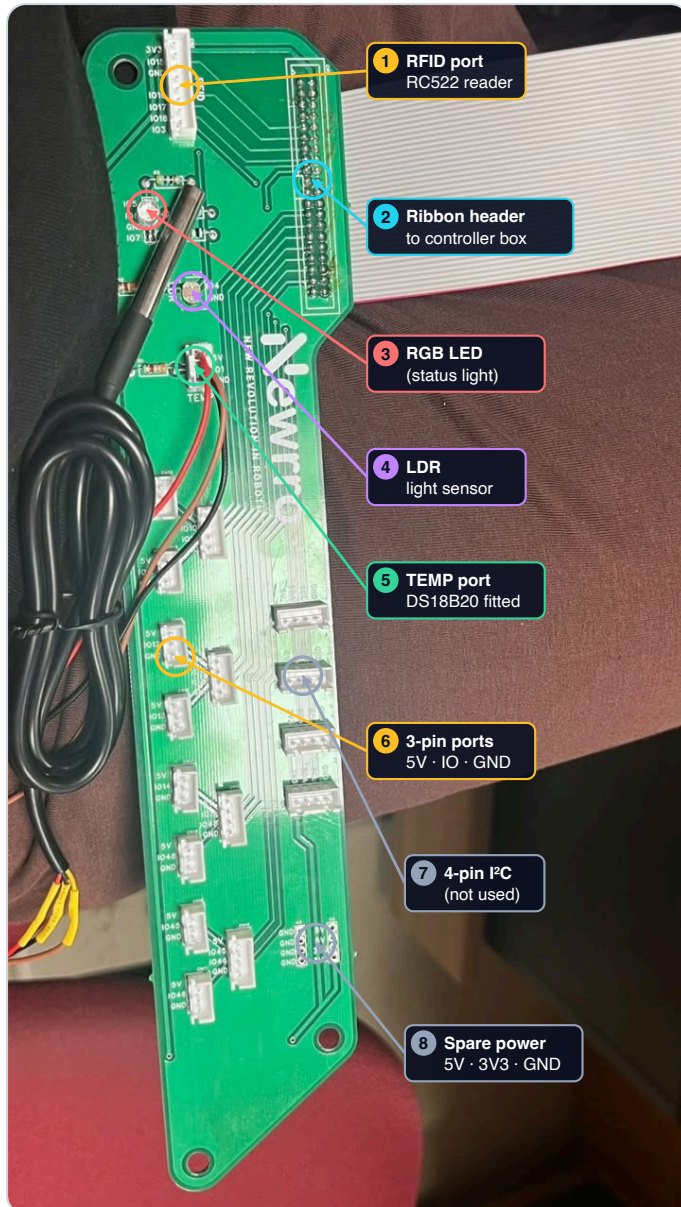
If your laptop has only USB-C

Use a USB-C ↔ USB-A adapter, or a USB-C ↔ USB-C *data* cable.

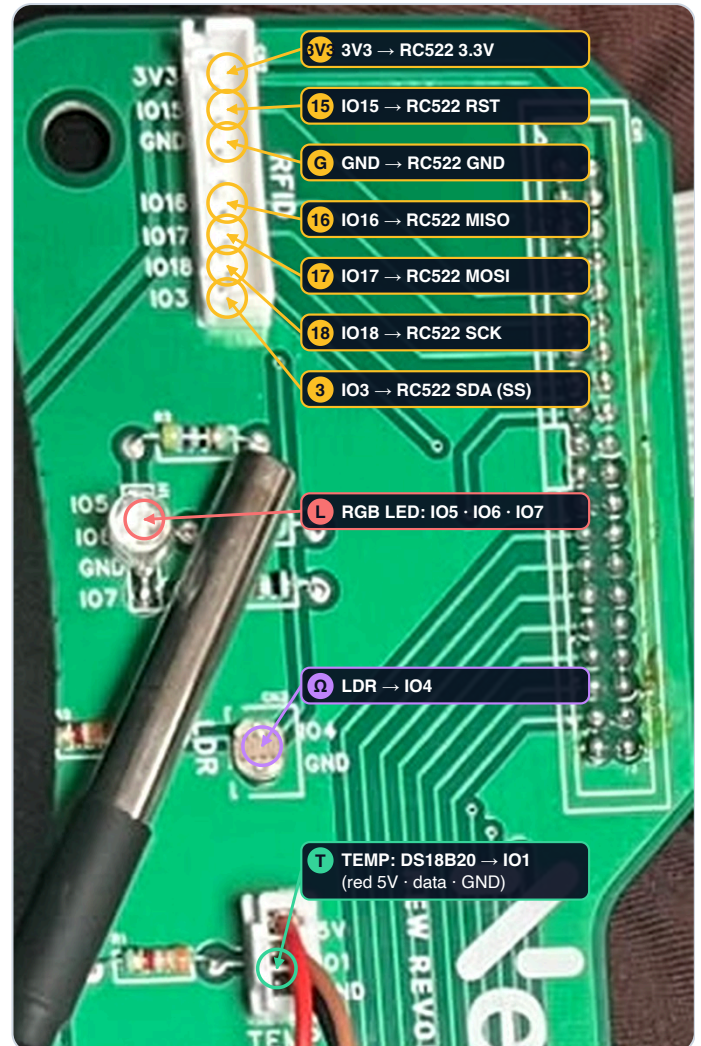
STEP 2

Get to know your sensor board

The long green Newrro board is where every sensor plugs in. It connects to the controller box with the grey **ribbon cable**. Everything is labelled on the board — here is what each area does in TRUSTMESH.



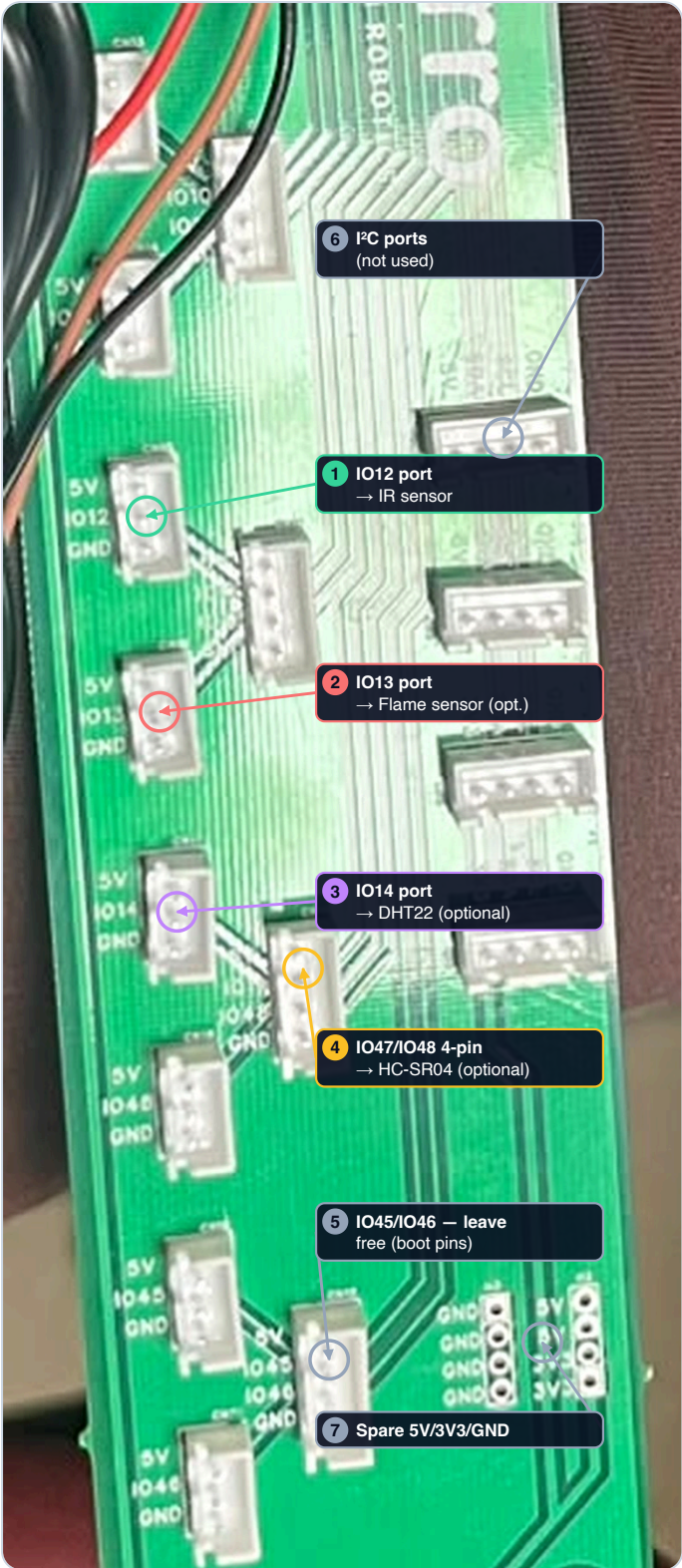
Overview of the Newrro sensor board (your photo).



Close-up: RFID port pin order (the 4th pin, IRQ, is not used).

The RFID port order matches the RC522's own pin order reversed (3.3V, RST, GND, IRQ, MISO, MOSI, SCK, SDA), so the SPI roles above are **inferred**. Step 7 checks it: the firmware prints `RC522 version 0x91/0x92 → ok` when the wiring is right.

The sensor ports & what goes where



Close-up of the 3-pin and 4-pin sensor ports.

Pin map used by the firmware

Part	Plug into	Signal	
RC522 RFID reader	8-pin RFID port (top)	3V3 · RST=IO15 · GND · (IRQ n/c) · MISO=IO16 · MOSI=IO17 · SCK=IO18 · SDA=IO3	Required
DS18B20 probe	TEMP port	5V · IO1 · GND	Required (already fitted)
LDR light sensor	soldered on board	IO4	Built in
RGB status LED	soldered on board	IO5 · IO6 · IO7	Built in
IR obstacle sensor	3-pin port 5V/IO12/GND	OUT → IO12	Required
Flame module	3-pin port 5V/IO13/GND	DO → IO13	Optional (FULL build)
DHT22	3-pin port 5V/IO14/GND	DATA → IO14	Optional (FULL build)

Firmware builds: `newrro_min` = RFID + DS18B20 + LDR + LED + IR. `newrro_full` adds flame (IO13) and DHT22 (IO14). The same map is in `docs/WIRING_TABLES.md`, generated from `firmware/pins/profiles.json`.

5-volt warning. These ports supply 5 V to the sensor, but the ESP32-S3 inputs only tolerate 3.3 V. Many IR/flame modules pull their output up to their supply. Step 4 shows how to measure it and what to do if it reads ~5 V.

Leave **IO45 / IO46** empty — they decide how the chip boots.

STEP 3

Start TRUSTMESH on the laptop

Do this before touching the hardware, so you can watch the kit come alive on the website.

- 1 Open Terminal in the project folder and start everything:

```
cd ~/trustmesh
npm run dev
```

Wait for “**TRUSTMESH is running**”. Keep this window open.

- 2 Open <http://localhost:3000>. You'll land on the **Live Demo**. Click 🦋 **My real hardware kit** at the top — it will say “*Real kit not connected*” for now. That's expected.

- 3 Install the firmware tool once (skip if `npm run doctor` says PlatformIO found):

```
python3 -m pip install --user platformio
echo 'export PATH=$PATH:$HOME/Library/Python/3.13/bin' >> ~/.zshrc && source ~/.zshrc
pio --version
```

Windows: `py -m pip install --user platformio` and use the `pio` path it prints.

Checkpoint: the website loads and the Live Demo shows the real-hardware switch.

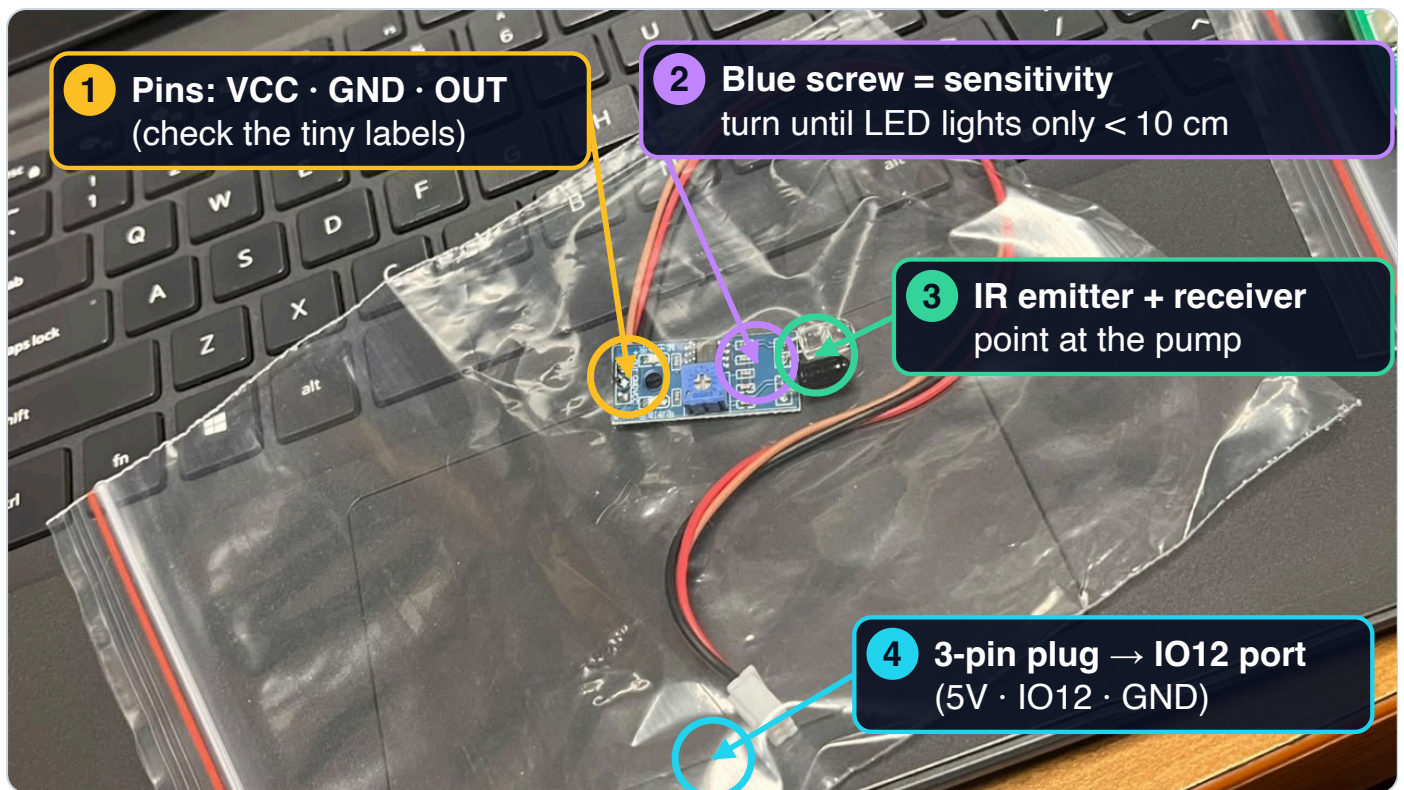
STEP 4

Plug the sensors into the board

Power off (switch at O, USB unplugged). Plug the parts in this order:

- 1 **DS18B20 probe** — already in the **TEMP** port in your photo. Leave it.
- 2 **RC522 RFID reader** → the 8-pin **RFID** port at the top of the board, matching the labels (3V3 to 3.3V, GND to GND...). Keep an RFID card/tag ready — it becomes the pump's identity tag.
- 3 **IR sensor** → the **IO12** 3-pin port (next page). Point it at where the pump/object will stand, about 5–10 cm away.
- 4 *Optional:* **flame sensor DO** → **IO13** port, **DHT22** → **IO14** port. With a flame sensor, use the `newrro_full` build; without it, use `newrro_min`.
- 5 Connect the **ribbon cable** between the board's header and the box's 40-pin socket. Match the **red-striped wire to pin 1** on both ends. Don't force it: the keyed plug only fits one way.

Connect & adjust the IR presence sensor



Your IR obstacle-avoidance module with its 3-wire cable.

Match the wires to the port — carefully

The board port order is **5V · IO12 · GND**. The module's order is **VCC · GND · OUT**. Make sure **VCC** → **5V**, **GND** → **GND**, **OUT** → **IO12**. If the cable's plug doesn't map that way, move the crimped wires in the plug housing (lift the small tab with a needle) or use female jumper wires.

Safety check (needs a multimeter, 1 minute): with the kit powered and nothing in front of the sensor, measure **OUT to GND**.

- $\leq 3.4 \text{ V}$ → safe, continue.
- $\sim 5 \text{ V}$ → don't connect OUT directly. Add a divider: $\text{OUT} \rightarrow 10 \text{ k}\Omega \rightarrow \text{IO12}$, and $\text{IO12} \rightarrow 20 \text{ k}\Omega \rightarrow \text{GND}$ (gives $\sim 3.3 \text{ V}$).

No multimeter? Many Newrro kits ship sensors matched to their board, but that's unverified here — at least keep the test short until you've measured.

Same check for the flame module ($\text{DO} \rightarrow \text{IO13}$). The firmware turns on internal pull resistors so an **unplugged** IR sensor reads “absent” and an **unplugged** flame sensor reads “flame” (fail-safe).

STEP 5

Connect the box & identify the chip

- 1 Plug the **USB-C** end into the controller box and the other end into the laptop. The box's POWER light should come on.
- 2 Find the port name:

```
cd ~/trustmesh
npm run bridge -- --list
```

You see	Meaning	Firmware build to use
/dev/cu.usbmodem... (Espressif, 303a)	ESP32-S3 native USB	newrro_full or newrro_min
/dev/cu.usbserial... / wchusbserial... (CH340/CP210x)	USB-to-serial chip	newrro_full_uart
Windows: COM3, COM5 ...	check Device Manager → Ports	as above
nothing new appears	charge-only cable, or the box needs its power switch on	try switch = I, another port/cable

- 3 Confirm the chip really is an **ESP32-S3** (replace the port):

```
cd ~/trustmesh/firmware
pio pkg exec -p tool-esptoolpy -- esptool.py --port /dev/cu.usbmodem1101 chip_id
```

Look for `Chip is ESP32-S3`. If it says plain `ESP32`, stop — the Newrro pin map needs changing (see Troubleshooting).

STEP 6

Write the device key & flash the firmware

- 1 Write the secret key of the registered device **ESP32-017** into the firmware (the file is never committed to git):

```
cd ~/trustmesh
npm run firmware:secrets -- --device ESP32-017
```

- 2 Build & upload (pick the build from step 5 · choose `newrro_min` if you have no flame sensor):

```
cd ~/trustmesh/firmware
pio run -e newrro_full -t upload --upload-port /dev/cu.usbmodem1101
```

“Failed to connect to ESP32”? Hold the **BOOT** button (if your box exposes one) while the upload starts, or unplug → hold BOOT → plug in → release. Close any other program using the port (Arduino IDE, serial monitor).

Checkpoint: [SUCCESS] at the end of the upload.

STEP 7

Test every sensor (diagnostic mode)

- 1 Open the serial monitor:

```
cd ~/trustmesh/firmware
pio device monitor -b 115200
```

You should see lines like:

```
# TRUSTMESH firmware - profile NEWRR0_FULL - device ESP32-017
# RC522 version 0x92 → ok
# DS18B20 devices: 1
# boot 3fa2c1d0 device ESP32-017 profile NEWRR0_FULL transport serial bridge
```

- 2 Type these commands (press Enter after each) and check the result:

Type	What should happen	If not
<code>!led r</code> / <code>!led g</code> / <code>!led b</code>	onboard LED turns red / green / blue	colours swapped → tell the maintainer; order IO5/6/7 is inferred
<code>!diag on</code>	a <code># DIAG ...</code> line every second with every reading	—
hold the RFID tag on the reader	<code>rfidPresent=true rfidUid=...</code>	RC522 FAULT → check 3.3 V & port seating
hand in front of IR sensor	<code>irPresent=true</code>	adjust blue screw; check OUT → IO12
hold the metal probe	<code>probeTempC</code> rises (~30 °C)	<code>null(fault)</code> → probe not seated in TEMP
cover the LDR with a finger	<code>ldrRaw</code> changes a lot	—
TV remote at flame sensor (FULL)	<code>flame=true</code> → LED red (interlock)	adjust its screw
<code>!only probe</code> · <code>!diag off</code>	show only one sensor · stop	—

“hello failed (-1)” lines are normal here: the website link isn't running yet (next step).

- 3 Close the monitor (`Ctrl+C`). Only one program can use the USB port at a time.

STEP 8

Link the kit to the website

- 1 In a second terminal (keep `npm run dev` running in the first):

```
cd ~/trustmesh
npm run bridge -- --port /dev/cu.usbmodem1101
```

You'll see `hello → 200` then `telemetry → 200` every 2 seconds.
- 2 On the website: **Engineer console** → **Devices** shows **ESP32-017 · online · REAL HW · serial**, auth **OK**. The onboard LED turns **green**.

Checkpoint: the Live Demo's real-hardware banner says “**• Real kit ESP32-017 connected via USB**”.

STEP 9

Run the Live Demo with real hardware

Open **Live Demo** → select  **My real hardware kit**. Each button now uses your physical kit and the asset **PUMP-017** (its rules accept **only real hardware** evidence).

Step on the website	What you do with the kit	What the judge sees
▶ Start the demo	Hold the RFID tag on the reader, object in front of the IR sensor.	First time: “Enrolled the RFID tag ...”. Without a flame sensor, the owner's rule is updated on-chain to <i>RFID + presence</i> (shown as a policy change).
👉 Send the pump to Technician #42	Keep tag + object in place.	Hand-over confirmed only after your sensors prove it. LED blinks blue during the check.
🔥 Trigger a fire alarm	Click, then within 90 s move the object away from the IR sensor (or TV remote at the flame sensor).	LED turns red · incident on the blockchain · pump LOCKED.
📦 Try to sell the locked pump	Nothing.	The contract refuses the sale.
🔧 Repair & inspect	Put the object back in front of the IR sensor, tag on the reader.	Technician repairs with fresh sensor proof → independent inspector approves → unlocked.
🕵️ Try to fake a sensor record	Nothing.	The fake is caught (fingerprints differ).

Wi-Fi mode & troubleshooting

Wi-Fi instead of USB (optional)

Put laptop and kit on the same 2.4 GHz Wi-Fi, set `API_HOST=0.0.0.0` in `.env`, then:

```
npm run firmware:secrets -- --device ESP32-017 --wifi "YourWiFi" --pass "password"
cd firmware && pio run -e newrro_full -t upload --upload-port <port>
```

Restart `npm run dev`. The bridge is then not needed. Guest/hotspot networks often block device-to-laptop traffic — USB is the reliable choice for judging.

Troubleshooting

Problem	Fix
Real-kit banner stays “not connected”	Is <code>npm run bridge</code> running? Serial monitor closed? Box switched on?
Bridge: <code>Resource busy</code> / <code>Access denied</code>	Close <code>pio device monitor</code> / Arduino IDE; unplug & replug.
Devices page: auth <code>BAD_MAC</code>	Key mismatch → rerun <code>npm run firmware:secrets</code> and re-upload.
“Hold the RFID tag ... press again”	The reader must see the tag within 30 s before “Start”.
Hand-over waits then fails	Verification needs tag and IR presence at the same moment; check Transfers page → “Last verification result”.
Alarms keep firing by themselves	The IR sensor loses the object (removal = alarm). Fix its position / sensitivity screw.
Chip is not ESP32-S3	The Newrro pin map doesn't apply; share a photo of the controller's chip and we'll add that board.
Nothing on serial after flashing	Wrong build for your USB type — try <code>newrro_full_uart</code> (or the reverse).

Judge-day checklist

- ☐ Laptop charged, sleep disabled, `npm run dev` running
- ☐ Kit powered, USB-C data cable in, `npm run bridge -- --port ...` running
- ☐ Devices page shows ESP32-017 **online · auth OK**
- ☐ RFID tag in hand, object on its spot in front of the IR sensor
- ☐ Live Demo → “My real hardware kit” → ► **Start the demo** done once before judges arrive
- ☐ Fallback ready: switch to 🟢 **Simulated sensor box** if anything misbehaves
- ☐ Public link (if used): tunnel running, link tested on your phone

Safety & honesty notes

- Use only low-voltage parts; no mains wiring. Keep water away from the electronics. For “fire”, use a TV-remote IR stimulus only — never real flame.
- RC522 is a 3.3 V part; ESP32-S3 inputs are 3.3 V — measure 5 V module outputs before connecting (Step 4).
- Pin functions for the Newrro board were read from your photos and are marked *inferred* where the silkscreen doesn't state the role (RFID SPI order, LED colour order). The diagnostic step verifies them.
- What the demo proves: the recorded data can't be changed without detection and the rules can't be bypassed. It does not prove the sensor itself was honest or calibrated, and RFID tags can be copied.

Generated from the TRUSTMESH repository (`scripts/build-hardware-guide.ts`). Pin source of truth: `firmware/pins/profiles.json` (profiles NEWRRO_MIN / NEWRRO_FULL).